

En esta tutorial aprenderemos a aplicar dos métodos clásicos de **aprendizaje no supervisado: K-Means y Clustering Jerárquico**. A diferencia de aprendices supervisados, aquí no contamos con etiquetas previas, el algoritmo es capaz de descubrir estructuras ocultas en los datos agrupando observaciones similares.

- El tutorial está organizado en dos grandes partes:
1. **K-Means Clustering**: Introducido sobre un dataset sintético y después sobre el dataset real NCND.
 2. **Hierarchical Clustering**: Aplicado sobre el dataset NCND con distintas métricas y métodos de enlace.
- Cada sección explica **qué se va a hacer**, **por qué se hace** y **qué significan los resultados**.

Parte 1 – K-Means Clustering

¿Qué es K-Means?

K-Means es uno de los algoritmos de clustering más populares. Su objetivo es particionar un conjunto de datos en **K grupos (clusters)** de modo que los puntos dentro de cada grupo sean lo más similares posible entre sí y lo más diferentes posible respecto a los demás grupos.

El algoritmo funciona de forma iterativa siguiendo estos pasos:

1. **Inicialización**: Se eligen **K** puntos como centroides iniciales (aleatoriamente o con K-Means++).
2. **Asignación**: Cada punto se asigna al centroide más cercano (distancia euclidiana).
3. **Actualización**: Se recalcula cada centroide como la media de los puntos asignados a él.
4. **Convergencia**: Se repite los pasos 2 y 3 hasta que los centroides no cambian significativamente.

1.1 Generación del Dataset Sintético

Comenzamos creando un dataset artificial con `make_blobs` de `sklearn`. Esto nos permite ver paso a paso cómo funciona K-Means antes de aplicarlo a datos reales.

Generamos **1000 muestras con 3 centros y 3 características**, lo que facilita la visualización en 2D.

```
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
import random as rd

# Generamos el dataset sintético con 1000 muestras, 3 centros y 3 features
X, y = make_blobs(n_samples=1000, centers=3, n_features=3, cluster_std=2)

# Creamos un DataFrame para facilitar la visualización de los datos
df = pd.DataFrame(columns=['Feature 1', 'Feature 2'])

print(f"Shape: {X.shape} del dataset generado")
display(df.head())

# Filtramos las columnas del dataset: (df.shape[0]) muestras, (df.shape[1]) características

# Creamos 3 filas del dataset generado:
Feature 1 Feature 2
0 0.065000 7.208602
1 1.186806 -0.007070
2 0.307010 -0.077057
3 0.708867 1.110270
4 0.718095 1.024941

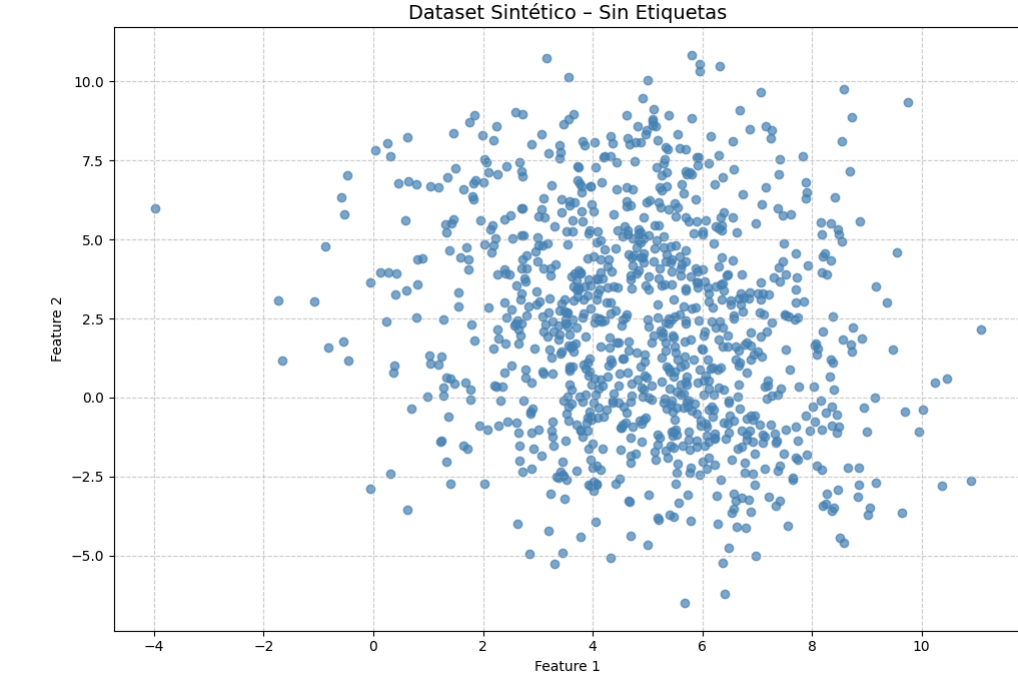
Dimensiones del dataset: 1000 muestras, 3 características
```

1.2 Visualización Inicial (Sin Etiquetas)

Antes de aplicar cualquier algoritmo, visualizamos los datos **sin etiquetas**. Esto simula la situación real del aprendizaje no supervisado: tenemos los puntos pero no sabemos a qué grupo pertenecen.

Observe que visualmente se pueden intuir agrupaciones, pero el algoritmo debe descubrirlas por sí solo.

```
plt.figure(figsize=(10, 7))
plt.scatter(df[['Feature 1', 'Feature 2']], color='tealblue')
plt.title('Dataset Sintético - Sin Etiquetas', fontsize=14)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()
```



1.3 Paso 1 de K-Means: Asignación Aleatoria Inicial

K-Means comienza asignando **etiquetas de cluster al azar** a cada punto. Esta asignación no es correcta, pero sirve como punto de partida para calcular los primeros centroides.

Este paso basta por qué la inicialización es importante: una asignación inicial muy mala puede llevar a una convergencia lenta o a un mínimo local subóptimo.

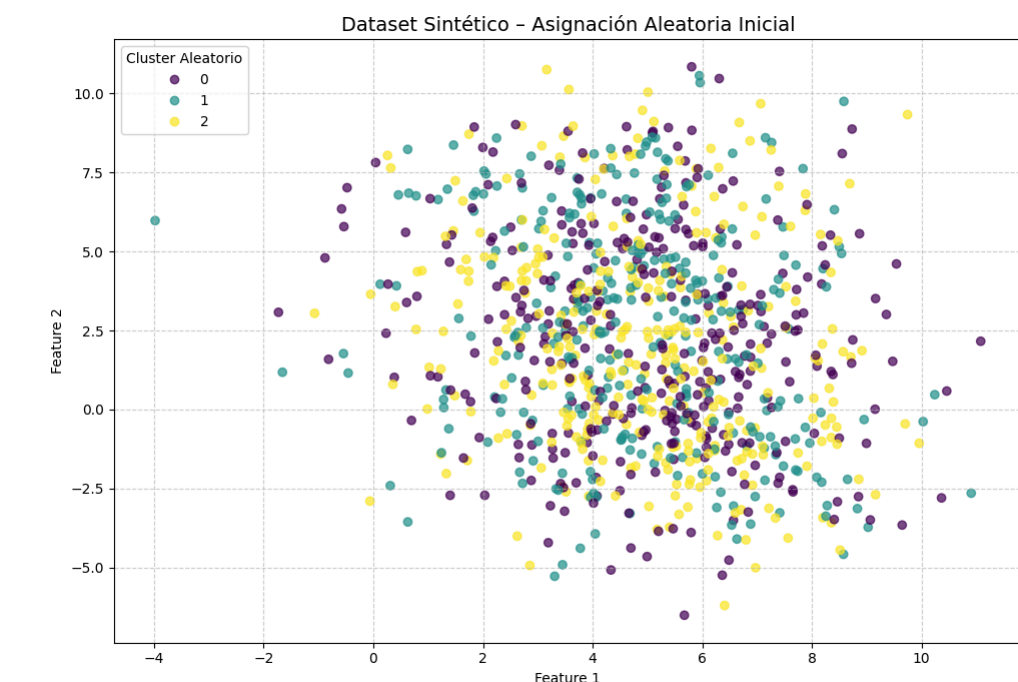
```
import numpy as np
n_m, samples = 100000

# Asignamos etiquetas aleatorias de cluster (0, 1 o 2) a cada punto
random_labels = np.random.randint(0, 3, size=samples)
df['random_labels'] = random_labels

print(f"Shape: {X.shape} y {df.shape[1]} del dataset con etiquetas aleatorias")
display(df.head())

# Creamos 3 filas del dataset con etiquetas aleatorias:
Feature 1 Feature 2 random_labels
0 0.065000 7.208602 0
1 1.186806 -0.007070 0
2 0.307010 -0.077057 2
3 0.708867 1.110270 0
4 0.718095 1.024941 1
```

```
plt.figure(figsize=(10, 7))
scatter = plt.scatter(df[['Feature 1', 'Feature 2']], s=100, color='violet', alpha=0.7)
plt.title('Dataset Sintético - Asignación Aleatoria Inicial', fontsize=14)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()
```



Como era de esperarse, la asignación aleatoria **no refleja la estructura real** del dataset. Los colores están mezclados sin ningún patrón coherente.

1.4 Paso 2 de K-Means: Cálculo de Centroides Iniciales

Una vez asignados los etiquetas, calculamos el **centroide de cada cluster** como la media de las coordenadas de todos los puntos que le fueron asignados.

Estos centroides serán el punto de partida para la siguiente iteración de reasignación.

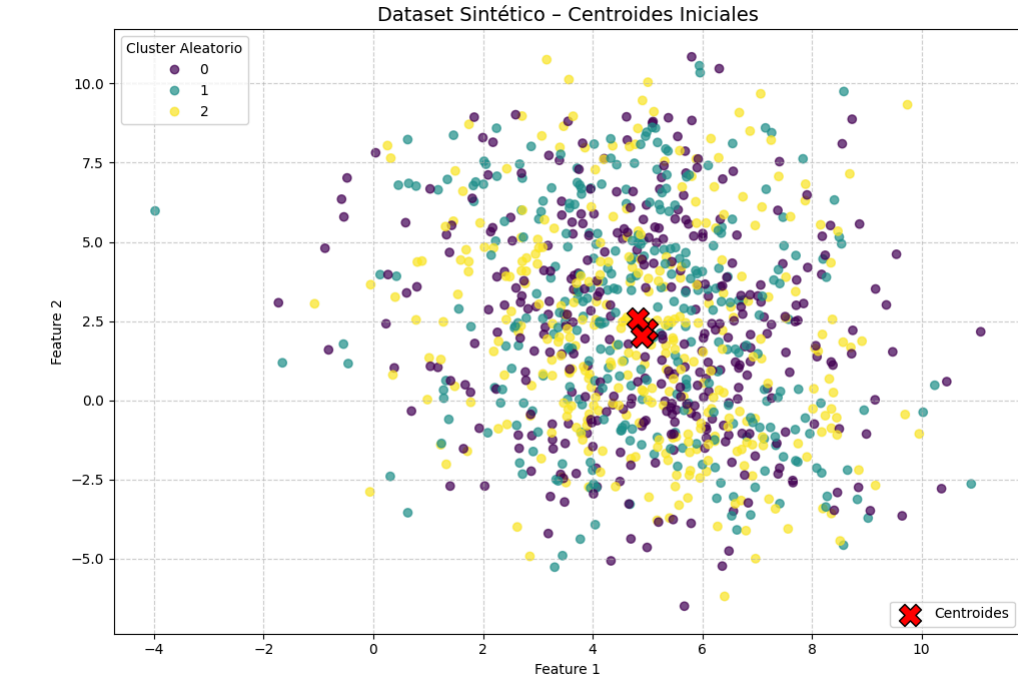
```
# Calculamos el centroide de cada cluster como la media de sus puntos
centroides = df.groupby('random_labels')[['Feature 1', 'Feature 2']].mean()

print(f"Centroides calculados para cada cluster aleatorio")
display(centroides)

# Centroides calculados para cada cluster aleatorio:
random_labels Feature 1 Feature 2
0 0.307010 2.202017
1 0.180010 0.000010
2 0.000000 0.000000
```

Notepad: [Generate code with Copilot](#) [See alternative plans](#)

```
plt.figure(figsize=(10, 7))
scatter = plt.scatter(df[['Feature 1', 'Feature 2']], s=100, color='violet', alpha=0.7)
plt.scatter(centroides[['Feature 1', 'Feature 2']], s=100, color='red', alpha=0.7)
plt.title('Dataset Sintético - Centroides Iniciales', fontsize=14)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()
```



Los centroides iniciales con 4 dígitos representan el "centro de masa" de cada grupo. Dado que la asignación fue aleatoria, los centroides están mal posicionados. La siguiente iteración los moverá hacia donde realmente están los grupos.

1.5 Paso 3: Reasignación de Puntos al Centroide Más Cercano

Ahora reasignamos cada punto al cluster cuyo centroide está **más cerca** usando distancia euclidiana. Esta es la operación fundamental que hace que K-Means converja.

La distancia euclidiana entre un punto **p** y un centroide **c** se calcula como:

$$d(p, c) = \sqrt{(p_1 - c_1)^2 + (p_2 - c_2)^2}$$

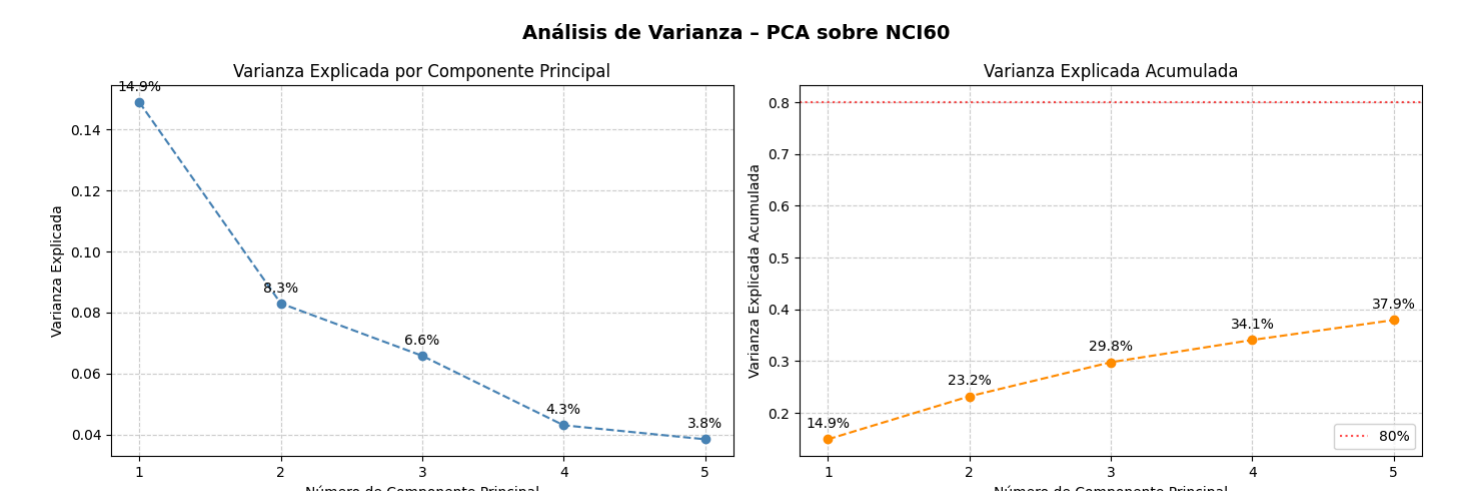
```
from scipy.spatial.distance import pdist

# Obtenemos las coordenadas actuales de los centroides y de los datos
current_centroids = centroides[['Feature 1', 'Feature 2']].values
features = df[['Feature 1', 'Feature 2']].values

# Calculamos la distancia euclidiana de cada punto a cada centroide
distancias = pdist(features, current_centroids, 'euclidean')

# Asignamos cada punto al centroide más cercano
new_clusters_assignment = np.argmin(distancias, axis=1)
df['random_labels'] = new_clusters_assignment

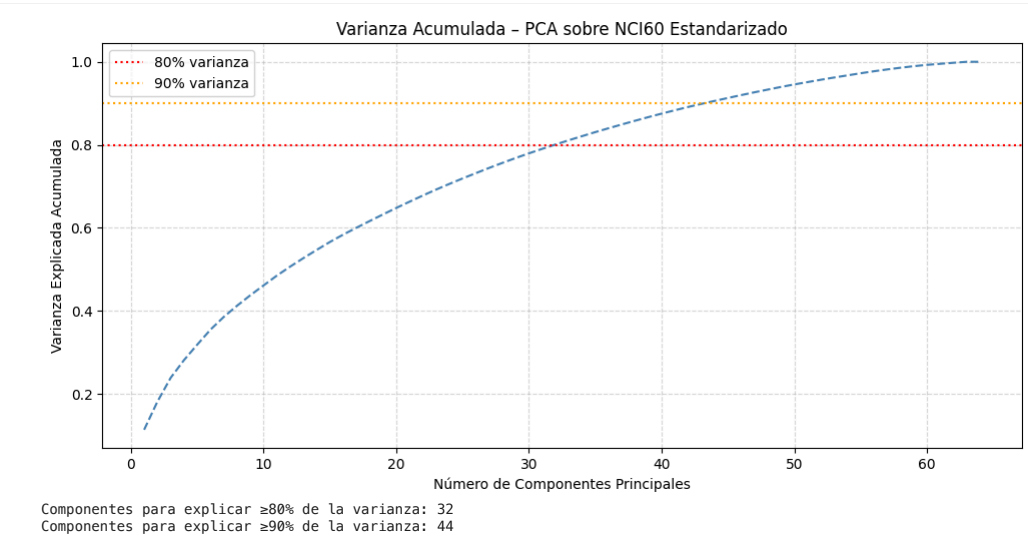
print(f"Shape: {X.shape} y {df.shape[1]} del dataset con las etiquetas de cluster actualizadas")
display(df.head())
```

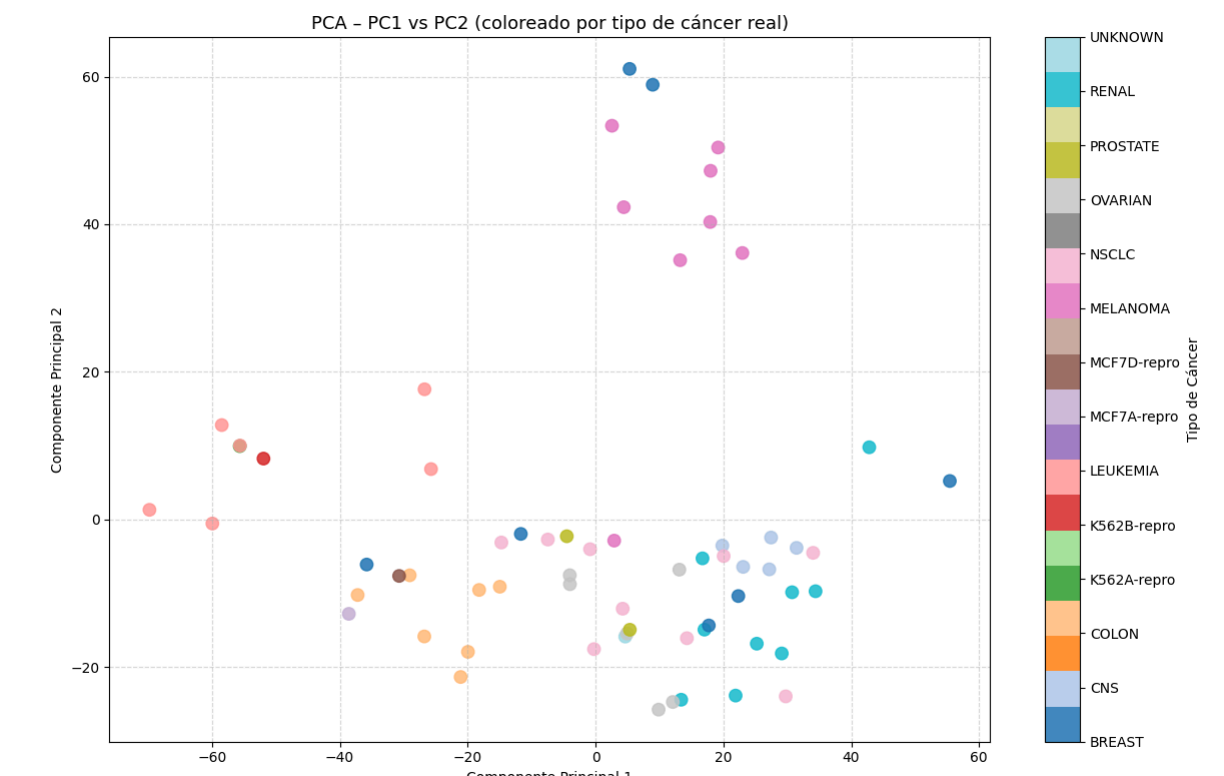
Parte 2 – Hierarchical Clustering (Clustering Jerárquico)

2.1 Carga y Preprocesamiento del NCI60

2.2 PCA + Varianza Explicada (sobre datos estandarizados)



2.3 Visualización PC1 vs PC2 (coloreado por tipo de cáncer real)



2.4 Clustering Jerárquico – Linkage 'Complete' + Métrica 'Euclídea'

```
from scipy.cluster.hierarchy import linkage, dendrogram

# Clustering jerárquico con estas 'completa' y métrica euclidiana
hierarchical_clustemas = linkage(X_dados, method='complete', metric='euclidean')

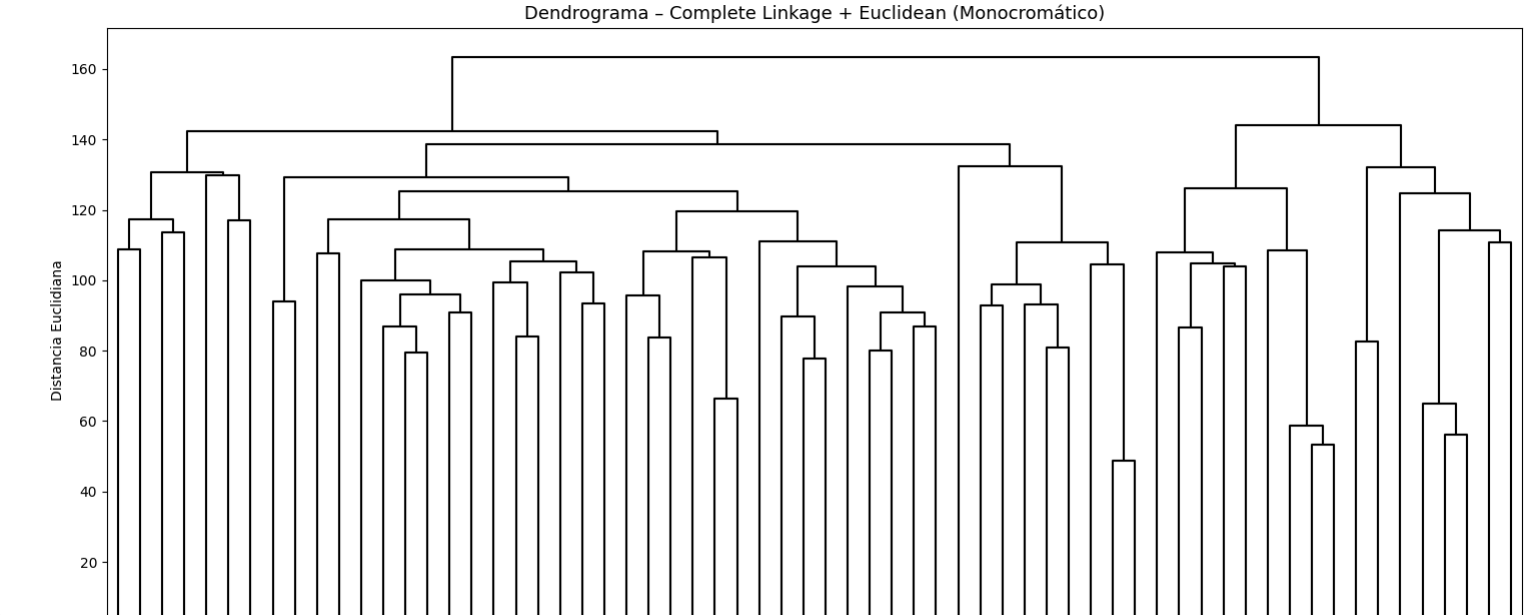
print("Matriz de linkage generada (hierarchical_clustemas.shape)")
print("Cada fila indica: [id_esc_usuario, id_esc_usuario2, distancia, num_hojasmas]")
print("Cada columna: 3 valores")
print(hierarchical_clustemas.shape[0])

# Matriz de linkage generada (h, 4)
# Cada fila indica: [id_esc_usuario, id_esc_usuario2, distancia, num_hojasmas]

# Primeros 3 valores
(h, n, m) = hierarchical_clustemas[0:3, 0:3]
print(h, n, m)
```

2.5 Dendrograma Monocromático (Complete + Euclidean)

El **dendrograma** es la representación visual del proceso de fusión jerárquica. La **altura del eje Y** indica la distancia a la que se fusionaron dos clusters: mayor altura = clusters más diferentes entre sí.



Para determinar el número óptimo de clusters, visualizamos el dendrograma cortado en distintos niveles. La línea roja **punteada** indica el umbral de corte.

```
# El umbral para obtener n clusters se obtiene de la distancia de la fusión (N - n)-ésima
```

```
plt.suptitle('Dendrogramas con Distintos M3odos de Clusters - Complete + Euclidean',
```

Dendrogramas con Distintos Números de Clusters - Complete + Euclidean

dos grupos son muy diferentes entre sí.

Ahora usamos la **métrica de correlación** en lugar de la euclídea. En genérica, la correlación es frecuentemente preferida porque

```
dendrogram1
  hierarchical_correlation,
```

Dendrograma - Complete Linkage + Métrica Correlación (Monocromático)

Método	Distancia en el espacio de features	Similitud de patrones
--------	-------------------------------------	-----------------------

anticorrelacionados]. Los clusters formados pueden ser distintos, reflejando grupos con patrones similares de expresión génica.

Exploramos el efecto de cambiar el método de **enlace (linkage)**. El método **average** define la distancia entre dos clusters como el

Companando con `complete` (máximo) y `single` (mínimo), el enlace `average` ofrece un término medio entre robustez y sensibilidad.

```
plt.figure(figsize=(15, 7))
ax = plt.gca()
```

```
color_threshold=0,
above_threshold_color='black'
```

Método	Parámetros clave	Ventajas	Desventajas
E-ikano	K (número de clusters), n_{init}	Rápido, escalable	Requiere definir K , sensible a inicialización y outliers

3.2 Conclusions

2. El clustering jerárquico revela la estructura multivariada de los datos sin necesidad de definir K . El dendrograma permite explorar

5. PCA como herramienta de visualización es indispensable cuando los datos tienen muchas dimensiones. Aunque se pierda información al reducir a pocas componentes, se gana interpretabilidad.

- ```
fig, axes = plt.subplots(1, 3, figsize=(21, 7))
```

3

```
bc,
leaf_rotation=90,
```

```
ax.set_title(title, fontsize=11, fontweight='bold')
```

```
plt.tight_layout()
plt.show()
```

```
print(" Clustering Jerárquico con distintos linkages y métricas.")
```



